

Lecture 04 | July, 2026

RAG and Agent Evaluation

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

RAG and Agent Evaluation

- **RAG and Agent Evaluation**
 - LLM as a judge
- Generating RAG Answers
 - Loading the data
 - Running RAG
 - Processing all questions

RAG and Agent Evaluation

So far, we evaluated retrieval. We checked whether search returns the document that should answer the question.

That is only the first step. A complete application still needs to produce a final answer. For RAG, this means checking the generated answer.

For agents, it also means looking at the tool calls the model made before producing the answer.

RAG evaluation checks the whole flow together.

This includes:

- search
- prompt
- LLM

RAG and Agent Evaluation

If the final answer is bad, the problem can come from any of these steps. The search might retrieve the wrong document, the prompt might omit important context, or the LLM might ignore the context.

In this part, we'll evaluate:

- RAG answers with an LLM judge
- Agent answers and tool-call trajectories

We'll use the agent from module 01, save the final answer, and also save the tool calls.

Then we can look at whether the answer is good and whether the trajectory looks reasonable.

LLM as a judge

- **RAG and Agent Evaluation**
 - **LLM as a judge**
- Generating RAG Answers
 - Loading the data
 - Running RAG
 - Processing all questions

LLM as a judge

For RAG and agent evaluation, we compare the generated answer with the original answer. The generated answer won't use the same words as the original. It's a generative model, so the phrasing will be different even when the meaning is the same.

This is why we use another LLM to do the comparison. We show the judge the question, the original answer, and the generated answer. Then we ask it to decide if they are semantically equivalent.

This approach is called LLM-as-a-judge.

The evaluating LLM is the judge. It classifies each answer as good or bad and explains its reasoning.

LLM as a judge

Asking the judge to explain why it made a decision generally produces better classifications than asking for just the verdict.

Next, we'll start with the RAG case and generate answers for the ground truth questions.

Generating RAG Answers

- RAG and Agent Evaluation
 - LLM as a judge
- **Generating RAG Answers**
 - Loading the data
 - Running RAG
 - Processing all questions

Generating RAG Answers

In the first part of this module, we evaluated search quality. We checked whether the right document appeared in the search results.

Now we evaluate the full RAG pipeline. For each generated question, we run RAG and save the answer produced by the LLM. Later, we'll compare this answer with the original FAQ answer.

This is the A->Q->A' setup:

- A = original answer in the FAQ
- Q = generated question from this answer
- A' = answer produced by our RAG system

If A' is close to A, the RAG system is doing a good job.

This is still offline evaluation.

We can compare A and A' because our questions came from FAQ records. For each question, we know which original answer it came from.

Loading the data

- RAG and Agent Evaluation
 - LLM as a judge
- **Generating RAG Answers**
 - **Loading the data**
 - Running RAG
 - Processing all questions

Loading the data

```
import pandas as pd
from ingest import load_faq_data, build_index

df_ground_truth = pd.read_csv("data/ground_truth-new.csv")
ground_truth = df_ground_truth.to_dict(orient="records")

documents = load_faq_data()

documents_llm = []

for doc in documents:
    if doc["course"] == "llm":
        documents_llm.append(doc)

documents = documents_llm
index = build_index(documents)
# Create a lookup table for the original FAQ documents:
doc_idx = {}

for doc in documents:
    doc_idx[doc["doc_id"]] = doc
```

Running RAG

- RAG and Agent Evaluation
 - LLM as a judge
- **Generating RAG Answers**
 - Loading the data
 - **Running RAG**
 - Processing all questions

Running RAG

```
from dotenv import load_dotenv  
from openai import OpenAI
```

```
load_dotenv()  
openai_client = OpenAI()
```

Running RAG

For this lesson, use `RAGWithUsage` from the evaluation utilities. It subclasses `RAGBase` from module 01, so it has the same `rag` method.

It stores token usage after each LLM call. Then we can calculate the total cost later.

It also uses the search boosts we selected in the search tuning lesson: `question=1.0`, `answer=2.0`, and `section=0.1`.

Running RAG

```
from evaluation_utils import RAGWithUsage
```

```
assistant = RAGWithUsage(  
    index=index,  
    llm_client=openai_client,  
)
```

Running RAG

For each question, RAGBase searches the FAQ, builds a prompt with the retrieved context, and asks the LLM to answer. We save the answer so the next lesson can judge it.

Run RAG for one question:

```
rec = ground_truth[0]
question = rec["question"]

answer_llm = assistant.rag(question)
answer_llm

# Check the cost of this call:
assistant.total_cost()
```

Running RAG

Get the original answer from the document ID:

```
doc_id = rec["document"]
original_doc = doc_idx[doc_id]
answer_orig = original_doc["answer"]
```

```
answer_orig
```

Now save both answers in one record:

```
rag_result = {
    "question": question,
    "answer_llm": answer_llm,
    "answer_orig": answer_orig,
    "document": doc_id,
}
```

```
rag_result
```

Processing all questions

- RAG and Agent Evaluation
 - LLM as a judge
- **Generating RAG Answers**
 - Loading the data
 - Running RAG
 - **Processing all questions**

Processing all questions

Create a function that processes one ground truth record:

```
def generate_rag_answer(rec):  
    question = rec["question"]  
    doc_id = rec["document"]  
    original_doc = doc_idx[doc_id]  
  
    answer_llm = assistant.rag(question)  
    answer_orig = original_doc["answer"]  
  
    result = {  
        "question": question,  
        "answer_llm": answer_llm,  
        "answer_orig": answer_orig,  
        "document": doc_id,  
    }  
  
    return result
```